



Exercices — Jour 1

React Native · Modules 1 & 2 : Introduction + Bases

Progression : chaque exercice s'appuie sur le précédent. Ne passez à l'exercice suivant qu'une fois le précédent validé par le formateur ou les tests.

Exercice 1 — Vérification de l'environnement (15 min)

Objectif : s'assurer que l'environnement de développement est fonctionnel.

Étapes

1. Vérifier les versions installées :

```
node --version      # doit être >= 18.x
npm --version       # doit être >= 9.x
git --version
watchman --version  # macOS/Linux uniquement
```

2. Installer `create-expo-app` et créer votre premier projet :

```
npx create-expo-app@latest FormationRN
cd FormationRN
npx expo start
```

3. Ouvrir l'application :

- Sur **émulateur Android** : appuyer sur `a` dans le terminal
- Sur **simulateur iOS** (macOS) : appuyer sur `i`
- Sur **appareil physique** : scanner le QR code avec Expo Go

✓ Critères de validation

- L'application affiche l'écran de démarrage Expo par défaut
- Le rechargement rapide fonctionne (modifier un texte dans `app/index.tsx` et sauvegarder)
- Aucune erreur rouge dans la console

● Exercice 2 — Explorer la structure du projet (10 min)

Objectif : comprendre l'organisation d'un projet Expo SDK 55.

Questions à compléter

Ouvrez les fichiers suivants et répondez :

Fichier	Rôle
<code>app.json</code>	?
<code>app/_layout.tsx</code>	?
<code>app/index.tsx</code>	?
<code>tsconfig.json</code>	?
<code>package.json</code>	?

1. Quelle est la valeur du champ "name" dans `app.json` ?
2. Quel composant est exporté par défaut dans `app/index.tsx` ?
3. Dans `package.json`, listez 3 dépendances directes (pas devDependencies).

✓ Critères de validation

- Le tableau est complété
- Les 3 questions ont une réponse précise

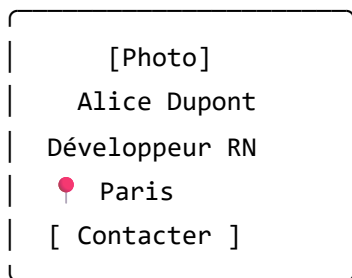
● Exercice 3 — Premier composant : Carte de présentation (25 min)

Objectif : créer un composant fonctionnel avec `View` , `Text` , `Image` et `StyleSheet` .

Consigne

Créez le fichier `components/CartePresentation.tsx` qui affiche :

- Une **photo de profil** circulaire (utilisez une image via URL)
- Un **prénom** en grand et en gras
- Un **rôle** en sous-titre (ex. : "Développeur React Native")
- Une **ville** avec une icône 📍
- Un **bouton** "Contacter" avec une couleur de fond



Code de départ

```
// components/CartePresentation.tsx
import { View, Text, Image, TouchableOpacity, StyleSheet } from 'react-native';

interface CartePresentationProps {
  prenom: string;
  nom: string;
  role: string;
  ville: string;
  photoUrl: string;
  onContact: () => void;
}

export default function CartePresentation(props: CartePresentationProps) {
  // À compléter
  return <View />;
}

const styles = StyleSheet.create({
  // À compléter
});
```

Bonus

- Ajouter une ombre portée (iOS : `shadowColor` , Android : `elevation`)
- Rendre la ville cliquable (ouvrir Google Maps)

✓ Critères de validation

- Le composant accepte toutes les props typées en TypeScript
- L'image est circulaire (`borderRadius` = moitié de la taille)
- Le bouton est stylisé (pas le `<Button>` natif)
- Pas d'erreur TypeScript

● Exercice 4 — useState : Compteur avancé (20 min)

Objectif : maîtriser `useState` et la mise à jour fonctionnelle du state.

Consigne

Créez `app/compteur.tsx` avec les fonctionnalités suivantes :

1. **Compteur** affiché en grand au centre
2. **3 boutons** : `-` , `Reset` , `+`
3. **Limite** : min 0, max 10 (les boutons se désactivent aux limites)
4. **Historique** : liste des 5 dernières valeurs sous le compteur
5. **Couleur dynamique** : le chiffre est vert si pair, rouge si impair

```
// Exemple de structure attendue
function CompteurScreen() {
  const [count, setCount] = useState(0);
  const [historique, setHistorique] = useState<number[]>([]);
  // ...
}
```

Contraintes techniques

- Utiliser la **mise à jour fonctionnelle** : `setCount(prev => prev + 1)`
- Ne jamais dépasser les limites (pas de valeurs négatives ni `> 10`)
- L'historique ne garde que les **5 dernières valeurs** (utiliser `slice`)

Bonus

- Ajouter un bouton "Annuler" qui restaure la valeur précédente
- Animer le changement de couleur avec `Animated`



Critères de validation

- Les boutons `-` et `+` sont désactivés aux limites
- L'historique se met à jour correctement et ne dépasse pas 5 entrées
- La couleur change selon la parité



Exercice 5 — `useEffect` : Minuteur (25 min)

Objectif : comprendre le cycle de vie avec `useEffect` et le `cleanup`.

Consigne

Créez `app/minuteur.tsx` : un minuteur de compte à rebours configurable.

Fonctionnalités :

1. Saisie de la durée initiale (TextInput numérique)
2. Boutons Démarrer , Pause , Réinitialiser
3. Affichage MM:SS (ex : 02:45)
4. Alerte ou vibration quand le minuteur atteint 0
5. Le minuteur continue en arrière-plan (AppState)

```
import { useState, useEffect, useRef } from 'react';
import { AppState } from 'react-native';

function MinuteurScreen() {
  const [duree, setDuree] = useState(60); // secondes
  const [restant, setRestant] = useState(60);
  const [actif, setActif] = useState(false);
  const intervalRef = useRef<ReturnType<typeof setInterval> | null>(null);

  // Formater en MM:SS
  const formater = (sec: number) => {
    const m = Math.floor(sec / 60).toString().padStart(2, '0');
    const s = (sec % 60).toString().padStart(2, '0');
    return `${m}:${s}`;
  };

  // À compléter : useEffect pour démarrer/arrêter l'intervalle
}
```

Points clés à maîtriser

- **Cleanup** du `setInterval` dans le `return` de `useEffect`
- Éviter les **memory leaks** (vérifier que le composant est toujours monté)
- **Dépendances** du `useEffect` correctement renseignées

✓ Critères de validation

- Pas de fuite mémoire (erreur React dans la console)
- Le `cleanup` est présent dans tous les `useEffect`

- Le format MM:SS est correct

● Exercice 6 — TypeScript avancé : Galerie de photos (35 min)

Objectif : combiner plusieurs hooks, TypeScript strict et composants réutilisables.

Consigne

Créez une galerie de photos avec filtres par catégorie.

Données à utiliser (API gratuite) :

<https://picsum.photos/v2/list?page=1&limit=30>

Structure attendue :

[Toutes] [Nature] [Architecture] [Portraits]

img	img	img
id:1	id:2	id:3

Types à définir :

```
interface Photo {  
  id: string;  
  author: string;  
  width: number;  
  height: number;  
  download_url: string;  
}
```

```
type Categorie = 'toutes' | 'nature' | 'architecture' | 'portraits';
```

Fonctionnalités :

1. Chargement depuis l'API avec état loading / error / data

2. Grille 2 colonnes avec `FlatList` et `numColumns={2}`
3. Barre de filtres horizontale (`ScrollView` horizontal)
4. Tap sur une photo → afficher en plein écran (`Modal`)
5. Afficher le nom de l'auteur sous chaque photo

Bonus

- Pull-to-refresh
- Pagination infinie (`onEndReached`)
- Skeleton loading (bibliothèque `moti`)



Critères de validation

- Typage TypeScript complet — aucun `any`
- Les 3 états (loading, error, data) sont gérés dans l'UI
- La `FlatList` utilise `keyExtractor` avec un identifiant stable
- Le `Modal` se ferme correctement (pas de fuite mémoire)

● Exercice 7 — Debug : Corriger une application cassée (20 min)

Objectif : utiliser React Native DevTools pour identifier et corriger des bugs.

Consigne

Le fichier ci-dessous contient **5 bugs**. Identifiez-les et corrigez-les.


```
// BuggyApp.tsx – NE PAS MODIFIER DIRECTEMENT : copier et corriger
import { useState, useEffect } from 'react';
import { View, Text, FlatList, Button } from 'react-native';

const BuggyApp = () => {
  // Bug 1
  let [items, setItems] = useState([]);

  // Bug 2
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/todos')
      .then(r => r.json())
      .then(data => setItems(data));
  });

  const ajouter = () => {
    // Bug 3
    items.push({ id: Date.now(), title: 'Nouveau' });
    setItems(items);
  };

  return (
    <View>
      Mes tâches : {items.length}
      <Button title="Ajouter" onPress={ajouter} />
      <FlatList
        data={items}
        // Bug 4
        renderItem={({ item }) => <Text>{item.title}</Text>}
      />
      <View>
        {/* Bug 5 */}
        Texte sans balise Text
      </View>
    </View>
  );
};
```

Réponse attendue

Complétez ce tableau :

#	Ligne	Bug	Correction
1	7	?	?
2	10-13	?	?
3	16-19	?	?
4	27	?	?
5	32	?	?